



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Formal Guided Test Sequence Optimization

Lorenzo Pattaro Zonta

29 May 2026

Introduction

Verification process faces significant challenges:

Introduction

Verification process faces significant challenges:

- # of test sequences grows exponentially with # of test cases.

Introduction

Verification process faces significant challenges:

- # of test sequences grows exponentially with # of test cases.
- Executing all possible sequences is often infeasible

Introduction

Verification process faces significant challenges:

- # of test sequences grows exponentially with # of test cases.
- Executing all possible sequences is often infeasible
- Identification of the most effective test sequences: crucial for efficient testing.

Agenda

1. Current Verification Approaches

2. Related Works

3. FGTSO Framework

4. Colors

5. Fonts

6. Block Environments

7. Special Frames

Agenda

– 1. Current Verification Approaches

2. Related Works

3. FGTSO Framework

4. Colors

5. Fonts

6. Block Environments

7. Special Frames

Current Verification Approaches

Formal verification:

Simulation-based verification:

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.

Simulation-based verification:

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

- High scalability and applicability.

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

- High scalability and applicability.
- Effectiveness rely on well-designed constraints.

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

- High scalability and applicability.
- Effectiveness rely on well-designed constraints.
- Biased constraints: miss critical scenarios.

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

- High scalability and applicability.
- Effectiveness rely on well-designed constraints.
- Biased constraints: miss critical scenarios.
- Strict constraints: increase debugging complexity.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions
- Challenges:

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions
- Challenges:
 - Resolve false alarms from formal simulation discrepancies.
 - Address BBOX constraints.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions
- Challenges:
 - Resolve false alarms from formal simulation discrepancies.
 - Address BBOX constraints.
- Integrate simulation and formal verification.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions
- Challenges:
 - Resolve false alarms from formal simulation discrepancies.
 - Address BBOX constraints.
- Integrate simulation and formal verification.
- Introduce abstract models to overcome limitations of BBOX.

Agenda

1. Current Verification Approaches

– 2. Related Works

3. FGTSO Framework

4. Colors

5. Fonts

6. Block Environments

7. Special Frames

Related Works

RISC-V Torture Test:

- Generate random instruction sequences to achieve faster coverage closure
- Lacks structured verification guidance: limited effectiveness in extreme corner cases

RISCV-DV:

- Support CRV (constraint random verification) and RISC V ISA features
- Reliance on randomness makes it difficult to cover boundary conditions

Fuzzing tools: cannot guarantee coverage of all possible scenarios.

Problems & Issues

- False alarms due to environmental discrepancies.
- BBOX issues by excessive complexity or large storage modules.

Agenda

1. Current Verification Approaches
2. Related Works
- 3. FGTSO Framework
4. Colors
5. Fonts
6. Block Environments
7. Special Frames

Overall architecture

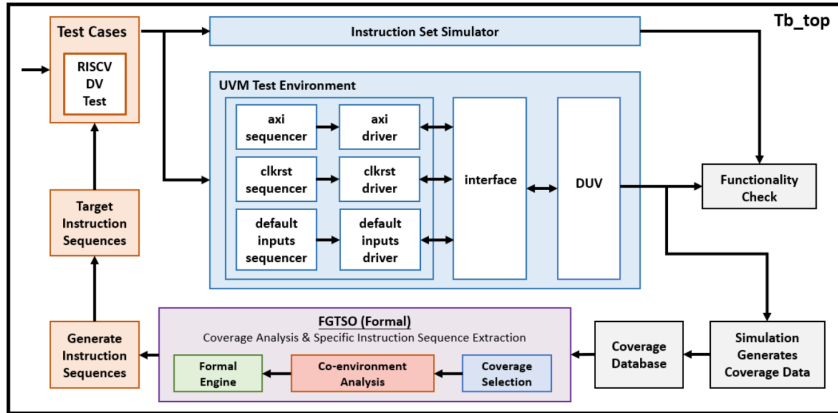


Figure: FGTSO Framework Architecture

Overview

1. Coverage Hole Selection

Overview

1. Coverage Hole Selection
2. Environment Consistency Check

Overview

1. Coverage Hole Selection
2. Environment Consistency Check
3. Formal Cover Property Generation

Overview

1. Coverage Hole Selection
2. Environment Consistency Check
3. Formal Cover Property Generation
4. Targeted Instruction Sequence Generation

FGTSO Workflow

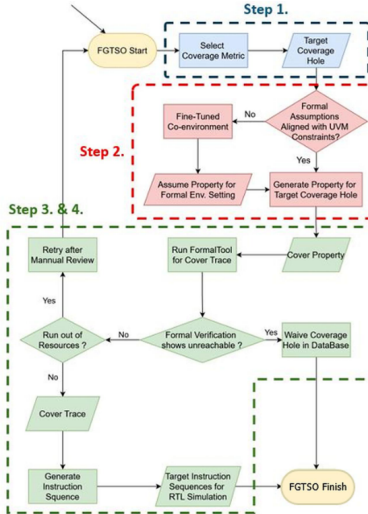


Figure: FGTSO Workflow

FGTSO Workflow - Step 1

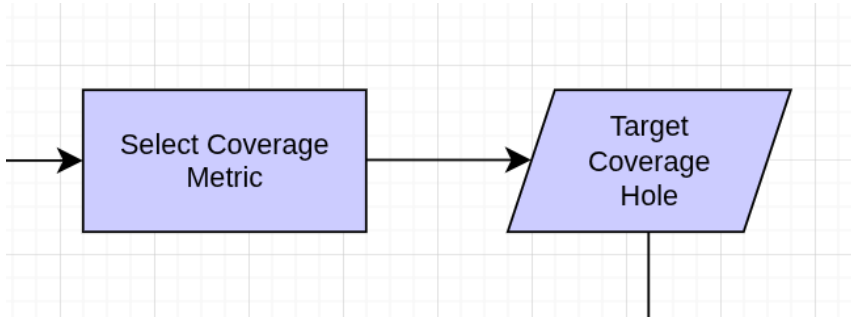


Figure: Step 1: Coverage Hole Selection

FGTSO Workflow - Step 1

- Priority: addressing most impactful coverage holes

FGTSO Workflow - Step 1

- Priority: addressing most impactful coverage holes
- MaxUncovd strategy: prioritizes selecting modules with lowest coverage, assuming that addressing the most uncovered areas yields the greatest verification impact.

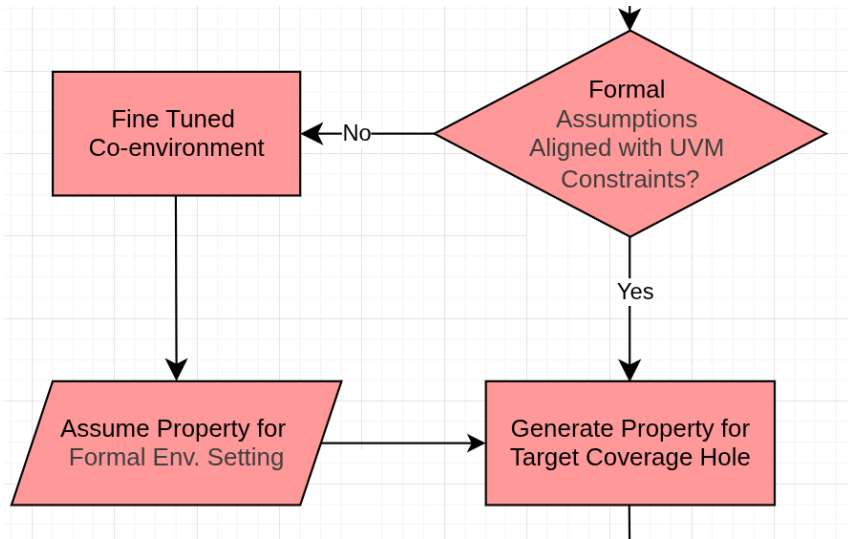


Figure: Step 2: Environment Consistency Check

- Misalignment detection is performed to determine consistency.

- Misalignment detection is performed to determine consistency.
- Formal verification generates instruction sequences targeting specific coverage holes, executed in the simulation.

- Misalignment detection is performed to determine consistency.
- Formal verification generates instruction sequences targeting specific coverage holes, executed in the simulation.
- Results from formal verification and simulation are inconsistent (fail to cover intended coverage holes): misalignment.

Three are the main causes of misalignment: PIs, BBOX and Insufficient RISC-V-DV constraints.

- Primary Inputs (PIs) such as reset, clock, interrupt request, rvalid signal for AXI protocol.

Three are the main causes of misalignment: PIs, BBOX and Insufficient RISC-V-DV constraints.

- Primary Inputs (PIs) such as reset, clock, interrupt request, rvalid signal for AXI protocol.
 - Treated as free nets: assume arbitrary values without constraints.
 - Incorrect coverage analysis: generated cover traces may fail to address coverage holes.
 - Unconstrained PIs can result in misleading coverage estimation. E.g.: test scenarios correspond to unreachable states by design (AXI protocol).
 - Formal verification may fail to generate meaningful instruction patterns.
 - Formal assumptions must be introduced to align formal verification with system behavior.

BBOX (Black Box) Constraints

- Definition: In EDA tools, modules with high storage requirements or complex computational logic are treated as BBOX, causing false alarms and verification gaps.

BBOX (Black Box) Constraints

- Definition: In EDA tools, modules with high storage requirements or complex computational logic are treated as BBOX, causing false alarms and verification gaps.
- BBOX modules cannot correctly drive outputs, they may become free nets, preventing module logic from driving its outputs.

BBOX (Black Box) Constraints

- Definition: In EDA tools, modules with high storage requirements or complex computational logic are treated as BBOX, causing false alarms and verification gaps.
- BBOX modules cannot correctly drive outputs, they may become free nets, preventing module logic from driving its outputs.
- Solution: Abstraction models preserve critical behavior while ensuring interaction with the system remains verifiable.

BBOX (Black Box) Constraints

- Definition: In EDA tools, modules with high storage requirements or complex computational logic are treated as BBOX, causing false alarms and verification gaps.
- BBOX modules cannot correctly drive outputs, they may become free nets, preventing module logic from driving its outputs.
- Solution: Abstraction models preserve critical behavior while ensuring interaction with the system remains verifiable.
- High Storage Example: Cache abstraction using SVA array to track indices. A single predefined tag replaces full tag tracking, reducing overhead while preserving cache functionality.

BBOX (Black Box) Constraints

- Definition: In EDA tools, modules with high storage requirements or complex computational logic are treated as BBOX, causing false alarms and verification gaps.
- BBOX modules cannot correctly drive outputs, they may become free nets, preventing module logic from driving its outputs.
- Solution: Abstraction models preserve critical behavior while ensuring interaction with the system remains verifiable.
- High Storage Example: Cache abstraction using SVA array to track indices. A single predefined tag replaces full tag tracking, reducing overhead while preserving cache functionality.
- Computation Example: Symbolic computation approach using SVA to approximate multiplier outputs, capturing input signals and using simplified results for consistency verification.

Cache Abstraction Model

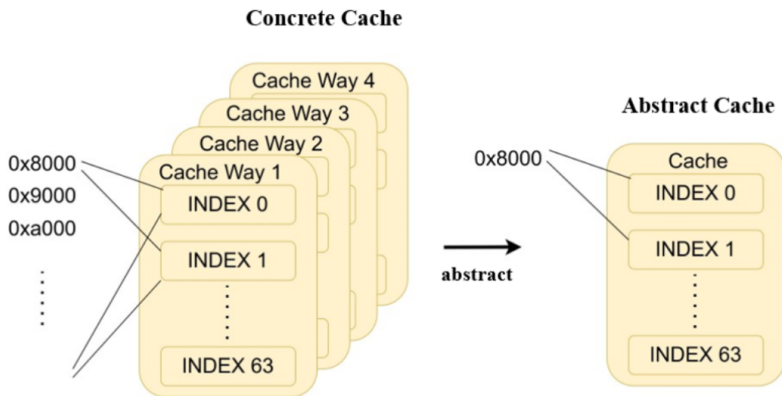


Figure: Cache Abstraction Model

Insufficient RISC-V-DV Constraints

- Problem: Formal verification may produce specialized memory data sequences that existing constraints in RISC-V-DV cannot properly handle.

Insufficient RISC-V-DV Constraints

- Problem: Formal verification may produce specialized memory data sequences that existing constraints in RISC-V-DV cannot properly handle.
- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.

Insufficient RISC-V-DV Constraints

- Problem: Formal verification may produce specialized memory data sequences that existing constraints in RISC-V-DV cannot properly handle.
- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.
- Solution: Modify constraints to apply them to addresses pointed to by 1 GB and 2 MB page table entries.

Insufficient RISC-V-DV Constraints

- Problem: Formal verification may produce specialized memory data sequences that existing constraints in RISC-V-DV cannot properly handle.
- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.
- Solution: Modify constraints to apply them to addresses pointed to by 1 GB and 2 MB page table entries.
- Implementation: Set X, W, and R bits to 0 to indicate that entries should point to the next level of the page table.

Insufficient RISC-V-DV Constraints

- Problem: Formal verification may produce specialized memory data sequences that existing constraints in RISC-V-DV cannot properly handle.
- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.
- Solution: Modify constraints to apply them to addresses pointed to by 1 GB and 2 MB page table entries.
- Implementation: Set X, W, and R bits to 0 to indicate that entries should point to the next level of the page table.
- Result: Coverage holes related to multi-level page table accesses can now be successfully filled.

Step 3 & 4: Formal Cover Property Generation

- Overview: Use formal cover properties to obtain cover traces and generate instruction sequences addressing target coverage holes.

Step 3 & 4: Formal Cover Property Generation

- Overview: Use formal cover properties to obtain cover traces and generate instruction sequences addressing target coverage holes.
- Property Definition: Step automatically extracts coverage hole expression and applies it to predefined cover templates.

Step 3 & 4: Formal Cover Property Generation

- Overview: Use formal cover properties to obtain cover traces and generate instruction sequences addressing target coverage holes.
- Property Definition: Step automatically extracts coverage hole expression and applies it to predefined cover templates.
- Formal Execution: Formal tool executes the cover property and derives the input patterns required to satisfy the condition.

Step 3 & 4: Formal Cover Property Generation

- Overview: Use formal cover properties to obtain cover traces and generate instruction sequences addressing target coverage holes.
- Property Definition: Step automatically extracts coverage hole expression and applies it to predefined cover templates.
- Formal Execution: Formal tool executes the cover property and derives the input patterns required to satisfy the condition.
- Cover Trace Generation: Generates corresponding cover trace from the formal verification results.

Step 3 & 4: Formal Cover Property Generation

- Overview: Use formal cover properties to obtain cover traces and generate instruction sequences addressing target coverage holes.
- Property Definition: Step automatically extracts coverage hole expression and applies it to predefined cover templates.
- Formal Execution: Formal tool executes the cover property and derives the input patterns required to satisfy the condition.
- Cover Trace Generation: Generates corresponding cover trace from the formal verification results.
- Sequence Extraction: Extract the instruction sequence from the trace to effectively cover the coverage hole.

Coverage Holes Analysis

SCORE	BRANCH	NAME	
86.05	86.05	wt_dcache_ctrl	179 MISS_REQ: begin
			180 miss_req_o = 1'b1;
87.50	87.50	fpnew_opgroup_fmt_slice	181
			182 if (req_port_i.kill_req) begin
			183 req_port_o.data_rvalid = 1'b1;
88.19	88.19	control_mvp	184 if (miss_ack_i) begin
			185 state_d = KILL_MISS;
88.24	88.24	controller	186 end else begin
			187 state_d = KILL_MISS_ACK;
90.00	90.00	scoreboard	188 end
			189 end else if (miss_replay_i) begin
90.00	90.00	bht	190 state_d = REPLAY_REQ;
			191 end else if (miss_ack_i) begin
91.67	91.67	fifo_v3	192 state_d = MISS_WAIT;
			193 end

Figure: Coverage Holes Identification

Cache Abstraction Model Codification

```
asm_tag_dcache_idle_0: assume property
(
  if (valid_tag_dcache[gen_cache_wt.i_cache_subsystem.i_wt_dcache.i_wt_dcache_mem.vld_addr])
    ##1 gen_cache_wt.i_cache_subsystem.i_wt_dcache.i_wt_dcache_mem.gen_tag_srams[0].i_tag_sram.rdata_o == 'h100000080000
  else
    ##1 gen_cache_wt.i_cache_subsystem.i_wt_dcache.i_wt_dcache_mem.gen_tag_srams[0].i_tag_sram.rdata_o == 'b0
);
```

Figure: SVA Implementation of Cache Abstraction

Formal Cover Property Example

```
test1: cover property
(
  gen_cache_wt.i_cache_subsystem.i_wt_dcache.gen_rd_ports[1].genblk1.i_wt_dcache_ctrl.state_q == 'h2 &&
  ~gen_cache_wt.i_cache_subsystem.i_wt_dcache.gen_rd_ports[1].genblk1.i_wt_dcache_ctrl.req_port_i.kill_req &&
  gen_cache_wt.i_cache_subsystem.i_wt_dcache.gen_rd_ports[1].genblk1.i_wt_dcache_ctrl.miss_replay_i
);
```

Figure: Cover Property Definition

Handling Coverage Challenges

- Unreachable Coverage Holes: First review whether constraints in both formal and simulation environments are overly restrictive.

Handling Coverage Challenges

- Unreachable Coverage Holes: First review whether constraints in both formal and simulation environments are overly restrictive.
- Waiving Unreachable Holes: If constraints are appropriate and hole is unreachable, it is waived; information still provided to designers for debugging reference.

Handling Coverage Challenges

- Unreachable Coverage Holes: First review whether constraints in both formal and simulation environments are overly restrictive.
- Waiving Unreachable Holes: If constraints are appropriate and hole is unreachable, it is waived; information still provided to designers for debugging reference.
- Resource Exhaustion: State space explosion is problematic for large-scale designs and limits formal tool capabilities.

Handling Coverage Challenges

- **Unreachable Coverage Holes:** First review whether constraints in both formal and simulation environments are overly restrictive.
- **Waiving Unreachable Holes:** If constraints are appropriate and hole is unreachable, it is waived; information still provided to designers for debugging reference.
- **Resource Exhaustion:** State space explosion is problematic for large-scale designs and limits formal tool capabilities.
- **Decomposition Strategy:** Complex properties are decomposed into smaller sub-properties to collect their cover traces independently.

Handling Coverage Challenges

- **Unreachable Coverage Holes:** First review whether constraints in both formal and simulation environments are overly restrictive.
- **Waiving Unreachable Holes:** If constraints are appropriate and hole is unreachable, it is waived; information still provided to designers for debugging reference.
- **Resource Exhaustion:** State space explosion is problematic for large-scale designs and limits formal tool capabilities.
- **Decomposition Strategy:** Complex properties are decomposed into smaller sub-properties to collect their cover traces independently.
- **Integration:** Integrate sub-property traces into comprehensive instruction sequence, ensuring manageable complexity while maintaining verification workability.

Final Outcome

```

main:
    addiw x10, x10, -27
    ld x24, 0(x2)
    slli x8, x8, 8
    addi x8, x2, 0
    csrrw x16, 0x7c1, x2
    lh x15, 4(x2)
    lw x9, 0(x8)
    lhu x3, 1192(x24)
    csrrsi x8, 0x7c1, 3
    lb x28, 1536(x9)
    beq x1, x2, next13632

next13632:
    slli x8, x8, 8
    lbu x0, -2047(x15)

```

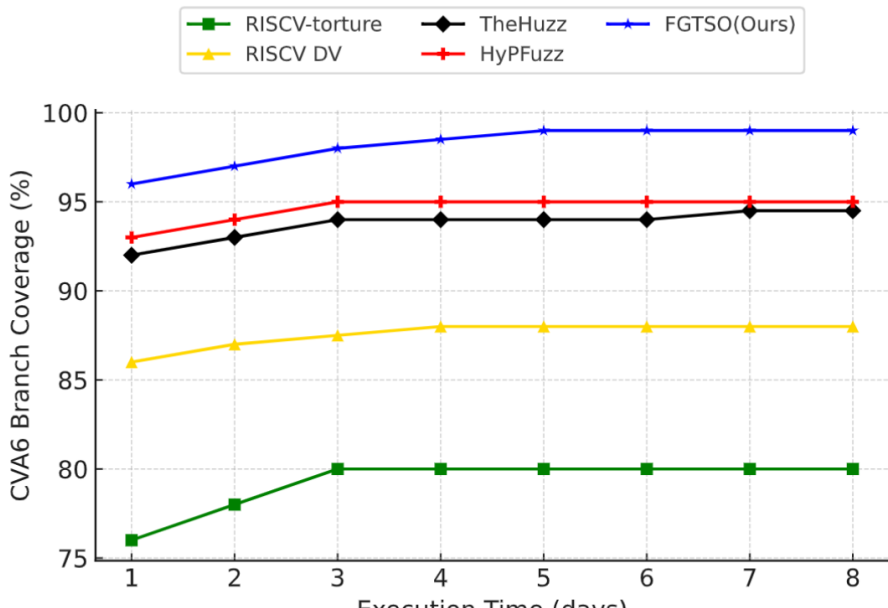
```

179     MISS_REQ: begin
180         miss_req_o = 1'b1;
181
182         if (req_port_i.kill_req) begin
183             req_port_o.data_rvalid = 1'b1;
184             if (miss_ack_i) begin
185                 state_d = KILL_MISS;
186             end else begin
187                 state_d = KILL_MISS_ACK;
188             end
189         end else if (miss_replay_i) begin
190             state_d = REPLAY_REQ;
191         end else if (miss_ack_i) begin
192             state_d = MISS_WAIT;
193         end

```

Figure: FGTSO Results: Coverage Achievement

Comparison with Other Frameworks



100% Coverage

Name	Score	Line	Toggle	Condition	Branch
uvmt_cva6_tb	100.00%	100.00%	100.00%	100.00%	100.00%
cva6_dut_wrap	100.00%	100.00%	100.00%	100.00%	100.00%
cva6_tb_wrapper_i	100.00%	100.00%	100.00%	100.00%	100.00%
i_cva6	100.00%	100.00%	100.00%	100.00%	100.00%
commit_sta...	100.00%	100.00%	100.00%	100.00%	100.00%
controller_i	100.00%	100.00%	100.00%	100.00%	100.00%
csr_regfile_i	100.00%	100.00%	100.00%	100.00%	100.00%
ex_stage_i	100.00%	100.00%	100.00%	100.00%	100.00%
gen_cache_...	100.00%	100.00%	100.00%	100.00%	100.00%
i_frontend	100.00%	100.00%	100.00%	100.00%	100.00%
id_stage_i	100.00%	100.00%	100.00%	100.00%	100.00%
issue_stage_i	100.00%	100.00%	100.00%	100.00%	100.00%

Figure: FGTSO: Achieving Complete Coverage

Background Patterns

The theme ships with 12 built-in background patterns. The easiest way to switch is via a theme option:

```
\usetheme[pattern=triangle]{wildcat}
```

Section pages automatically use the same pattern as the title page. Available pattern options:

- **facet** — NU facets (default)
- **facetrand** — NU facets (alternate)
- **triangle** — triangle mosaic
- **triangleiso** — isometric triangles
- **trianglemesh** — wireframe mesh
- **squares** — square grid
- **circles** — circles
- **hexraise** — raised hex clusters
- **hexrand** — random hex shading
- **network** — network grid
- **quilt** — quilt rings
- **plain** — solid color

See **wildcat-gallery.pdf** for a visual preview of all patterns.

Agenda

1. Current Verification Approaches

2. Related Works

3. FGTSO Framework

– 4. Colors

5. Fonts

6. Block Environments

7. Special Frames

Colors

The theme has a few Northwestern-specific colors defined, which you can use in your slides. These are:

- nupurple
- nupurple90
- nupurple80
- nupurple70
- nupurple60
- nupurple50
- nupurple40
- nupurple30
- nupurple20
- nupurple10
- nupurple160
- nupurple150
- nupurple140
- nupurple130
- nupurple120
- nupurple110
- nurichblack
- nubrightgreen
- nubrightteal
- nubrightblue
- nubrightyellow
- nubrightorange
- nubrightred
- nudarkgreen
- nudarkteal
- nudarkblue
- nudarkyellow
- nudarkorange
- nudarkred

Modifying the Primary Color

Pass a **color** option to `\usetheme` — the theme auto-generates all lighter and darker shades automatically:

```
\usetheme[color=nudarkblue]{wildcat}
```

You can combine **color** and **pattern** in a single line:

```
\usetheme[color=nudarkblue, pattern=hexraise]{wildcat}
```

The **nu*** colors from the previous slide are all valid choices, as are the standard xcolor built-ins (**red**, **blue**, **cyan**, etc.). The alert and example colors (**wcalerted**, **wcexample**) are not affected and can be overridden separately.

Modifying the Primary Color: Custom RGB

The **color** option accepts any *named* color — not raw RGB values. If your brand color isn't already defined, declare it first:

```
\definecolor{myblue}{RGB}{0,53,107}  
\usetheme[color=myblue]{wildcat}
```

When you pass a color via the **color** option, that color becomes available to use as **wcprimary** in your slides, and the theme auto-generates the lighter and darker shades as **wcprimary10–40** and **wcprimary110–140**, respectively. These are also available as named colors you can use in your slides, and the theme uses them internally for block backgrounds and pattern fills.

The auto-generated shades follow a simple scale: **wcprimary10–40** mix the base color with white (lighter tints), and **wcprimary110–140** mix it with black (darker shades). These are sufficient for most use cases — if you need precise per-shade control, see the Advanced Customization section at the end of this document.

Using Colors in Graphs

Python

```
import matplotlib.pyplot as plt
import pandas as pd
plt.style.use('ggplot')
# Create colors
wcprimary = (78/255,42/255,132/255)
df = pd.read_stata("source/graphs/auto.dta")
plt.scatter(df["weight"], df["mpg"], color=wcprimary)
plt.xlabel("Weight")
plt.ylabel("MPG")
plt.savefig("source/graphs/plot-python.pdf")
plt.show()
```


Using Colors in Graphs

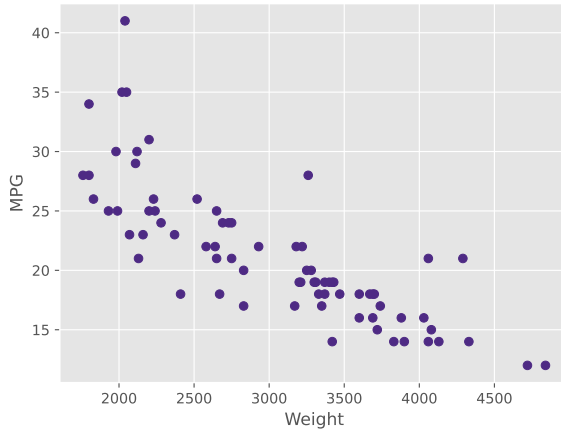


Figure: Python Graph Example

Using Colors in Graphs

R

```
library(ggplot2)
library(haven)
# Load data
df <- read_dta("source/graphs/auto.dta")
# Define custom RGB colors
wcprimary <- rgb(78/255, 42/255, 132/255)

# Plot mpg against weight, with marker color wcprimary
plot <- ggplot(df, aes(x = weight, y = mpg)) +
  geom_point(color = wcprimary) +
  labs(x = "Weight", y = "MPG")

# Save the plot to a file with a specific size
ggsave("source/graphs/plot-R.pdf", plot, width = 8, height = 5)
```

Using Colors in Graphs

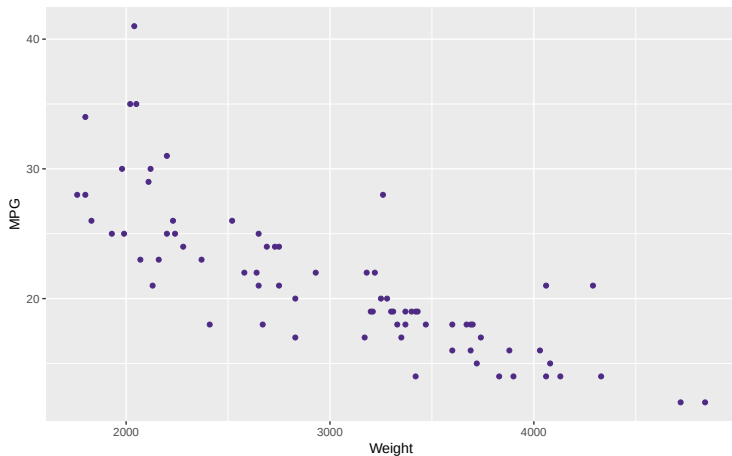


Figure: R Graph Example

Using Colors in Graphs

Stata

```
ssc install schemepack, replace
sysuse auto, clear
local wcprimary "78 42 132"

twoway (scatter mpg weight, mcolor("`wcprimary'")), ///
      scheme(gg_tableau)

graph export "source/graphs/plot-stata.pdf", replace
```

Using Colors in Graphs

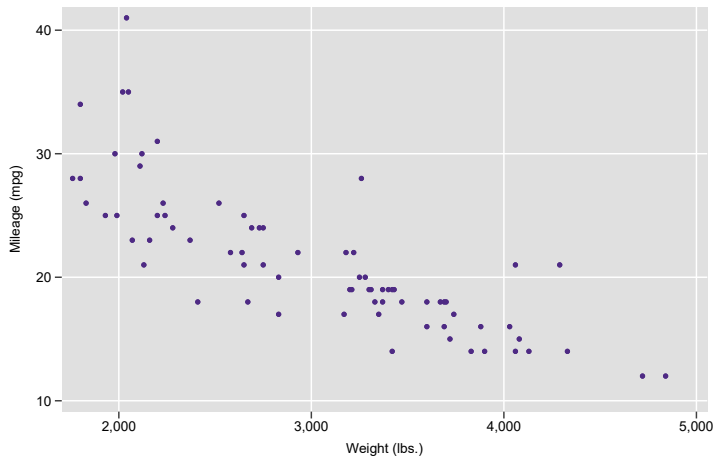


Figure: Stata Graph Example

Agenda

1. Current Verification Approaches

2. Related Works

3. FGTSO Framework

4. Colors

– 5. Fonts

6. Block Environments

7. Special Frames

Fonts: Default (Fira Sans)

By default, the theme uses Fira Sans — a clean, geometric sans-serif that ships with TeX Live and MiKTeX and is pre-installed on Overleaf. No font files need to be downloaded or bundled:

```

\usetheme{wildcat}           % Fira Sans loads automatically
  
```

Overleaf: works out of the box.

MiKTeX: auto-installs on first compile.

TeX Live: if the font is not found, run `tlmgr install fira` in your terminal, then recompile.

Requires XeLaTeX or LuaLaTeX — PDFLaTeX will fail.

Fonts: Noto Sans

For another bundled, Overleaf-friendly option, use **font=noto**. It uses Noto Sans and mirrors the same weight structure as the default Fira setup:

```

\usetheme[font=noto]{wildcat}
  
```

This is a good default when you want broad Unicode coverage and don't want to upload font files. As with the other theme fonts, compile with XeLaTeX or LuaLaTeX.

Fonts: Poppins + IBM Plex Sans

For a closer match to Northwestern's recommended free fonts, use **font=poppins**. Download Poppins and IBM Plex Sans from Google Fonts and place them in a **fonts/** folder next to your **.tex** file:

```

\usetheme[font=poppins]{wildcat}

```

```

your-presentation/
  your-slides.tex
  fonts/
    Poppins/
      Poppins-Medium.ttf      Poppins-MediumItalic.ttf
      Poppins-SemiBold.ttf    Poppins-SemiBoldItalic.ttf
      Poppins-Light.ttf       Poppins-LightItalic.ttf
      Poppins-Regular.ttf     Poppins-Italic.ttf
    IBM_Plex_Sans/
      IBMPLexSans-Light.ttf    IBMPLexSans-LightItalic.ttf
      IBMPLexSans-Regular.ttf  IBMPLexSans-Italic.ttf

```

For proprietary Northwestern brand fonts (Campton + Akkurat Pro), see the Advanced Customization section. [Appendix](#)

Font Style: Examples

Here is what the body font will look like under normal usage:

- Regular
- *Italic*
- Bold
- ***Bold Italic***
- Alert
- *Alert Italic*
- Math:

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n$$

Slide Numbering Options

The **numbering** option controls the footer style on normal slides:

<code>\usetheme[numbering=fraction]{wildcat}</code>	% default: n/N
<code>\usetheme[numbering=number]{wildcat}</code>	% slide number only
<code>\usetheme[numbering=progressbar]{wildcat}</code>	% bottom progress bar
<code>\usetheme[numbering=none]{wildcat}</code>	% no numbering

The count excludes the title page. The **progressbar** mode fills the bottom bar using **wcprimary** and leaves the remainder at 10% opacity. Section pages, title slides, standout slides, **[plain]** frames, and all appendix slides do not show numbering.

Breadcrumb Options

The **breadcrumb** option controls whether normal frame titles show the current section and subsection above the title:

```

\usetheme[breadcrumb=none]{wildcat}  % default
\usetheme[breadcrumb=on]{wildcat}
  
```

Breadcrumbs appear only after the first section starts. They are hidden on title, plain, section, and standout slides.

Agenda

1. Current Verification Approaches
2. Related Works
3. FGTSO Framework
4. Colors
5. Fonts
- 6. Block Environments
7. Special Frames

Blocks

Primary Block

The standard Beamer **block** environment uses the material-style outlined box with a floating title label.

Alert Block

alertblock uses the theme alert color.

Example Block

exampleblock uses the theme example color.

Alternate: Pattern Blocks

Pattern Block

If you want to use the main background pattern inside a `tcolorbox`, the theme also includes `pblock`, `palertblock`, and `pexampleblock`.

Pattern Alert Block

Patterned alert block.

Pattern Example Block

Patterned example block.

Alternate: Custom Block Colors

Custom Block

This block uses a custom color.

Custom Pattern Block

This patterned block uses a custom color.

Agenda

1. Current Verification Approaches

2. Related Works

3. FGTSO Framework

4. Colors

5. Fonts

6. Block Environments

7. Summary

Sections

`\section{}` groups slides and automatically inserts a section title slide. Pass `sectionpage=none` to `\usetheme` to disable them:

```
\usetheme[sectionpage=none]{wildcat}
```

Pass `agendastyle=pattern` to keep the 60% agenda panel white and show the active background pattern in the remaining 40%:

```
\usetheme[agendastyle=pattern]{wildcat}
```

Subsections

The normal `\subsection{}` command creates subsections for the agenda slide and the expanded current-section view on section divider slides.

The `[plain]` option produces a fully blank frame with no header, footer, or background. Use it for title cards, full-bleed figures, or transition slides:

```
\begin{frame}[plain]{}  
...  
\end{frame}
```

Standout Slides

The `\standout{}` command creates a full-pattern slide with centered white text — useful for section breaks, closing slides, or emphasis. No manual formatting needed:

```
\standout{Appendix}
```

Appendix

Advanced: NU Brand Fonts (Bundled)

Northwestern affiliates can use the official brand fonts — Campton (headings) and Akkurat Pro (body). These are proprietary; obtain them from northwestern.edu/brand. Bundle the OTF files in a **fonts/** folder and use:

```
\usetheme[font=nu]{wildcat}
```

```
your-presentation/
  your-slides.tex
  fonts/
    Campton/
      Campton Medium.otf    Campton Light.otf
    Akkurat/
      AkkuratProLight-Regular.otf  AkkuratProLight-Italic.otf
      AkkuratPro-Regular.otf      AkkuratPro-Bold.otf
```

Advanced: NU Brand Fonts (System-Installed)

If Campton and Akkurat Pro are already installed as system fonts (visible in Word, etc.), no **fonts/** folder is needed. Use:

```
\usetheme[font=installed]{wildcat}
```

fontspec loads the fonts directly from the OS font registry — no **fonts/** folder needed. This is convenient for local compilation, but won't work on Overleaf or when sharing with collaborators who don't have the fonts installed. In those cases, use **font=nu** with bundled OTF files as described on the previous slide.

Advanced: Fine-Grained Color Control

The auto-generated shades are a good starting point, but for precise brand colors you can override individual shades manually. Redefine any of the **wcprimary*** colors *after* **\usetheme**:

```
\usetheme{wildcat}                % or with color= option
\definecolor{wcprimary}{RGB}{0,53,107} % Yale Blue
\definecolor{wcprimary140}{RGB}{0, 34, 70}
\definecolor{wcprimary130}{RGB}{0, 40, 80}
\definecolor{wcprimary120}{RGB}{0, 45, 91}
\definecolor{wcprimary110}{RGB}{0, 50, 102}
\definecolor{wcprimary40}{RGB}{153, 174, 196}
\definecolor{wcprimary30}{RGB}{179, 194, 211}
\definecolor{wcprimary20}{RGB}{204, 215, 225}
\definecolor{wcprimary10}{RGB}{230, 235, 240}
```

Only the shades actually used by the theme need redefining: **wcprimary10–40** (block bodies) and **wcprimary110–140** (facet pattern segments).

Advanced: Custom Background Pattern (I)

To create your own pattern, define a command that draws TikZ commands filling the slide. The command should take one argument — the color — so that the theme can reuse it in different contexts. Here is a minimal example:

```
\newcommand{\mypattern}[1]{  
  \fill[color=#1] (0,0) rectangle (\paperwidth,\paperheight);  
}
```

The pattern command is called inside a **tikzpicture** environment whose bounding box already covers the full slide, so you only need to provide the drawing commands — no `\begin{tikzpicture}` wrapper is needed.

Any valid TikZ commands work. The built-in patterns each take one color argument and use `\paperwidth` / `\paperheight` to scale to the slide dimensions automatically.

Advanced: Custom Background Pattern (II)

Once your pattern command is defined, register it with the theme using two lines in your preamble *after* `\usetheme{wildcat}`:

```
\renewcommand{\bgpattern}{\mypattern{wcprimary}}  
\renewcommand{\bgpatternblock}[1]{\mypattern{#1}}
```

Two commands are needed because the theme uses the pattern in two different ways:

- `\bgpattern` — title page, section slides, and standout frames. The color is fixed, so you bake it in when registering.
- `\bgpatternblock{color}` — pattern block headers (`\pblock`, `\palertblock`, `\pexampleblock`), where each block supplies its own color at render time.

Advanced: Custom Background Pattern (III)

The 12 built-in pattern definitions in `beamerinnerthemewildcat.sty` (lines 31–386) are a useful reference and starting point.

`\bgpatternplain` (line 384) is the simplest — a single `\fill` command — and is a good template to copy and modify. The other patterns show progressively more complex TikZ techniques: coordinate scaling, `\foreach` loops, random seeds, scopes, and drop shadows.

The `\bgpatternblock` variant used in `pblock` headers is typically identical to `\bgpattern`, just registered separately so the theme can pass the block's color through as an argument.

Advanced: Community Background Patterns

A pattern template and example patterns are available in the **bg-gallery/** folder. To use one, add a single **\input** line to your preamble *after* **\usetheme{wildcat}**:

```
\usetheme{wildcat}  
\input{bg-gallery/bg-template}    % swap in any pattern file
```

Use **\input{}**, not **\include{}** — pattern commands must be set in the preamble, and **\include** only works in the document body.

Want to contribute? Copy **bg-gallery/bg-template.tex**, replace the drawing commands with your own TikZ, and open a pull request at github.com/aarondwolf/wildcat. Community patterns are welcome!